Reinforcement Learning

Lecture Notes

Carlos Cotrini
ETH Zürich

June 12, 2026

These notes accompany the interactive visualization *Repair or Replace*, available at

https://sml-fs26.github.io/classic_rl/repair-or-replace/

The concepts developed below map one-to-one onto its scenes.

1 Motivation

Many decisions a manager faces are sequential. What is decided today changes what must be decided tomorrow, and the consequences unfold over weeks and months rather than at once. The outcomes are uncertain on top: the same decision may turn out well one time and badly the next. Picture the scenario behind these notes. You operate a delivery business with a single van, and each week you make exactly one call about that machine: keep running it and collect the week's profit, at the risk of a breakdown and of gradual wear; repair it, paying for a week in the shop so that it fails less often afterwards; or replace it outright, a large expense that buys a fresh start. Every option trades a short-term cost against a long-term payoff, and chance interferes at every step.

Reinforcement learning is a framework for computing decision strategies that maximize the reward accumulated over time in precisely such problems. One running example, the repair-or-replace problem of the visualization, carries everything.

2 Markov decision processes

The first step is a precise model of the decision problem. The standard model is the *Markov decision process*.[1]

Definition 2.1 (Markov decision process)**.** A *Markov decision process* (MDP) is a quadruple, a package of four ingredients, (S, A, T, γ), where S is the set of *states* the system can be in; A is the set of *actions* available to the decision maker; T is the *transition dynamics*, which describes how the system reacts when an action is taken; and $\gamma \in [0, 1]$, the *discount factor*, is a number between 0 and 1. (γ is the Greek letter gamma.)

The sets S and A are defined a priori: they are fixed in advance, part of the problem description, and writing them down is the modeler's first task. The transition dynamics is then defined on top of them. It is a *randomized function* T that consumes a state s and an action a and produces a new state s' and a reward r:

$$(s', r) = T(s, a).$$

[1] The name records that the next state depends only on the current state and action, not on the past.

In words: feed the current state and the chosen action into the dynamics, and out come the state the system lands in next and the reward, a number measuring the immediate gain or loss of that step.

Remark 2.2. The transition dynamics is *random*: feeding the same state and action (s, a) in twice may produce different new states and different rewards.[2]

The discount factor γ is just a number between 0 and 1; its use will be explained later.

Example 2.3 (the repair-or-replace MDP)**.** You manage a one-van delivery fleet; the visualization calls the van *Old Bessie*. Each week the van sits at one of four wear levels, and you make exactly one call:

$$S = \{\text{HEALTHY}, \text{WORN}, \text{SHAKY}, \text{FAILING}\}, \qquad A = \{\text{RUN}, \text{SERVICE}, \text{REPLACE}\}.$$

In words: four wear levels, from factory-fresh to nearly dead, and three possible calls (SERVICE is the "repair" of the title). Rewards are weekly net money, in plain units. The numbers governing RUN are:

state	weekly profit	breakdown odds
HEALTHY	+95	2%
WORN	+72	8%
SHAKY	+40	28%
FAILING	+16	55%

Under RUN the van drives the route and earns the profit in the table, unless it breaks down, with the odds in the table: the week then books a single net reward equal to the profit minus a breakdown cost of 280 (from HEALTHY: $95 - 280 = -185$), and the van ends the week at FAILING. Even without a breakdown, wear tends to slip by one or two levels over the weeks. Under SERVICE the van spends the week in the shop: reward -50, no deliveries, and the wear often improves; from WORN, the van returns to HEALTHY 95% of the time. Under REPLACE the reward is -130, and the next week starts with a brand-new van at HEALTHY, whatever the old one looked like. The dynamics is random in the sense of Remark 2.2: feeding (WORN, RUN) into T may return (SHAKY, $+72$) one week and (FAILING, -208) another. There is no final week (the fleet runs indefinitely), and the visualization fixes $\gamma = 0.9$.

3 Policies and trajectories

An MDP describes the world the manager operates in; it does not say how to operate. That is the job of a policy.

Definition 3.1 (policy)**.** A *policy* defines how to act at each state. Formally, a policy is a function $\pi\colon S \to A$: it takes a state s and returns the action $\pi(s)$ to be played whenever the system is in s. (π is the Greek letter pi.)

Executing a policy on the MDP, week after week, generates a record of everything that happened. That record is the central object of these notes.

Definition 3.2 (trajectory)**.** The *trajectory* of a policy π is the sequence

$$s_1,\ a_1,\ r_1,\ s_2,\ a_2,\ r_2,\ \ldots,\ s_i,\ a_i,\ r_i,\ \ldots$$

produced by executing π on the MDP starting from state s_1. In words: states, actions, and rewards alternate, and the subscript counts the weeks. The mechanics are as follows. The first

[2] Scene "You run the fleet" of the interactive visualization lets you sample these dynamics.

action results from applying the policy to the first state, $a_1 = \pi(s_1)$. The first reward and the second state result from applying the transition dynamics to that pair, $(s_2, r_1) = T(s_1, a_1)$. Then $a_2 = \pi(s_2)$, the dynamics produce $(s_3, r_2) = T(s_2, a_2)$, and so on without end.

Remark 3.3. The trajectory is *random*: starting again from the same s_1 and executing the same policy may produce a different trajectory, because the transition dynamics is random (and so, possibly, is the policy, if one allows policies that randomize their choices).

Example 3.4 (a trajectory of the fleet)**.** Consider the playbook π with $\pi(\text{HEALTHY}) = \text{RUN}$, $\pi(\text{WORN}) = \text{SERVICE}$, and $\pi(\text{SHAKY}) = \pi(\text{FAILING}) = \text{REPLACE}$. One execution of π from $s_1 = \text{HEALTHY}$ produced[3]

$$\begin{aligned} &s_1 = \text{HEALTHY}, && a_1 = \text{RUN}, && r_1 = +95, \\ &s_2 = \text{WORN}, && a_2 = \text{SERVICE}, && r_2 = -50, \\ &s_3 = \text{HEALTHY}, && a_3 = \text{RUN}, && r_3 = +95, \\ &s_4 = \text{HEALTHY}, && a_4 = \text{RUN}, && r_4 = +95, \end{aligned}$$

after which $s_5 = \text{WORN}$ and the walk continues, one week per step, forever. In words: the van started healthy and earned 95 but slipped to worn; a 50 service restored it; two profitable runs followed; by week five it was worn again. Remark 3.3 is at work here: another execution of the same π from the same s_1 might open with a first-week breakdown, $s_1 = \text{HEALTHY}$, $a_1 = \text{RUN}$, $r_1 = -185$, $s_2 = \text{FAILING}$.

4 The goal: accumulating reward

A policy is good if it produces trajectories that accumulate as much reward as possible. The first task is to make "accumulate" precise.

Definition 4.1 (cumulative reward)**.** Take one particular trajectory $s_1, a_1, r_1, s_2, a_2, r_2, \ldots$. Its *cumulative reward*, also called its *return*, is

$$G_1 \;=\; r_1 + \gamma\, r_2 + \gamma^2 r_3 + \cdots + \gamma^{\,i-1} r_i + \cdots . \tag{4.1}$$

In words: add up the rewards of all weeks, forever (that is what the dots mean), scaling the reward of week i by $\gamma^{\,i-1}$: week one counts in full, week two is scaled by γ, week three by γ^2, and so on. (G stands for *gain*.)

Here the discount factor plays its very important role: it specifies how much we care about future rewards. With γ close to 0, the scaling factors die out almost immediately and only near-term rewards matter; with γ close to 1, they fade slowly and rewards far in the future count almost as much as immediate ones. Managers will recognize the device: it is the discounting of future cash flows familiar from net-present-value calculations, and, as with the NPV of a perpetuity, the shrinking factors keep the never-ending total finite. With the visualization's $\gamma = 0.9$, next week's reward counts 90% of this week's, the week after 81%, and so on.[4] For the trajectory of Example 3.4, formula (4.1) gives $G_1 = 95 + 0.9 \cdot (-50) + 0.9^2 \cdot 95 + 0.9^3 \cdot 95 + \cdots = 95 - 45 + 76.95 + 69.255 + \cdots$.

The same accounting can be started at any week, not only the first. For an arbitrary week i, define G_i as the cumulative reward from week i on:

$$G_i \;=\; r_i + \gamma\, r_{i+1} + \gamma^2 r_{i+2} + \cdots . \tag{4.2}$$

[3] Scene "The trajectory" of the interactive visualization replays such a tape week by week.

[4] Scene "Return over the van's life" of the interactive visualization has a slider for γ.

In words: G_i is what the trajectory accumulates from week i onward, with the discounting restarted there. Comparing (4.2) at weeks i and $i+1$ yields a recurrence that carries the rest of these notes:

$$G_i \;=\; r_i + \gamma\, G_{i+1}. \tag{4.3}$$

In words: what is accumulated from week i on equals this week's reward plus γ times what is accumulated from week $i+1$ on.

Remark 4.2. G_i is *random*. Because of the randomness of the transition dynamics, G_i comes out differently after every execution of the policy starting from state s_i.

A quantity that changes from run to run cannot be maximized directly; we average it instead.

Definition 4.3 (expectation)**.** The *expectation* $\mathbb{E}[G_i]$, read "the expected value of G_i," is the average of G_i over a sufficiently large set of trajectories: execute the policy many times, always starting from the same first state s_1 (for the fleet: a HEALTHY van), record the value of G_i of each execution, and average the records.

We can now state the ultimate objective of reinforcement learning: *find the policy π that maximizes* $\mathbb{E}[G_1]$, the policy whose trajectories accumulate, on average, the most reward. A policy achieving this maximum is called *optimal*. Note the contrast with G_i itself: $\mathbb{E}[G_i]$ is a single real number that does not depend on the trajectory, because the averaging has removed the randomness. Therefore $\mathbb{E}[G_i]$ is *not* random.

5 The Q-function and the Bellman equation

How can the best policy be discovered? The classical route goes through an auxiliary function that scores every pair of a state and an action.

Definition 5.1 (the Q-function)**.** For a state s and an action a, $Q^\star(s,a)$ is the cumulative reward one can expect when executing an optimal policy from state s, where the *first* action taken there is a. In notation,

$$Q^\star(s,a) \;=\; \mathbb{E}\big[\, G_i \;\big|\; s_i = s,\; a_i = a \,\big].$$

In words: the expected value of G_i conditioned on s_i being s and a_i being a (the vertical bar is read "given that"); that is, the average of G_i over many executions, counting only those executions that are in state s at week i and take action a there as a one-off, even when an optimal policy would choose differently, acting optimally afterwards. (The star in $Q^\star$ marks optimality.)

Which week i? It does not matter: the fleet has no final week, so it looks the same from week 3 as from week 30, and every week gives the same average.

Remark 5.2. $Q^\star(s,a)$ is different from $\mathbb{E}[G_i]$. The number $\mathbb{E}[G_i]$ averages over *all* executions of the policy, whatever state they happen to be in at week i. $Q^\star(s,a)$ restricts the average to the executions that pass through s at week i and are made to play a there, acting optimally afterwards. And while $\mathbb{E}[G_i]$ is a single number, $Q^\star$ is a whole table of numbers, one per state-action pair: for the fleet, $4 \times 3 = 12$ entries.

The table is worth the trouble because the optimal policy can be read off it: at state s, the optimal action is the action a for which $Q^\star(s,a)$ is largest. We write $\pi^\star(s)$ for this action, and $\pi^\star$ for the resulting optimal policy.

It remains to compute the table. Suppose an optimal policy produced the trajectory $\ldots, s_i, a_i, r_i, s_{i+1}, a_{i+1}, \ldots$. Averaging both sides of the recurrence (4.3), $G_i = r_i + \gamma\, G_{i+1}$, over many such executions, it must be the case that the *Bellman equation* holds:

$$\boxed{Q^\star(s_i, a_i) \;=\; \mathbb{E}\big[\, r_i + \gamma\, Q^\star(s_{i+1}, a_{i+1}) \,\big]}\,. \tag{5.1}$$

In words: the value of this week's state and action equals this week's reward plus γ times the value of next week's state and action, averaged over the randomness of the week. Replacing G_{i+1} by its average $Q^\star(s_{i+1}, a_{i+1})$ is legitimate: averaging in two rounds (first among the executions that share next week's state and action, then across the week's outcomes) gives the same result, just as a company-wide average can be computed branch by branch.

We can therefore find $Q^\star$ by solving the Bellman equations, one equation per state-action pair, twelve for the fleet. Since the optimal policy at s_{i+1} picks the action with the largest $Q^\star$-value, the right-hand side of (5.1) involves the same unknowns $Q^\star(s, a)$ as the left-hand side, so this is a system of equations in the entries of the table. The visualization solves it iteratively; its $Q^\star$-scorecard and Bellman-check scenes display the solution for the fleet at $\gamma = 0.9$.[5] Two entries tell the story: $Q^\star(\textsc{shaky}, \textsc{service}) = 126.4$, while $Q^\star(\textsc{shaky}, \textsc{replace}) = 149.9$. The optimal playbook replaces a van that still starts every morning.

6 SARSA: learning from the logbook

Solving the Bellman equations requires knowing the odds. The average in (5.1) runs over the randomness of the week, and computing an average over outcomes means knowing how likely each outcome is: the transition dynamics T itself. In the visualization those odds are printed on the fleet sheet; in reality nobody hands them to you, because no manual lists the breakdown odds for *your* van on *your* routes. And for realistic fleets the table of states explodes anyway (count odometer readings, age, weeks since the last service, route load, then forty vans, and the visualization's illustrative tally reaches some four million states for a single van and half a billion scorecard cells for a modest fleet).[6] What the manager does have is the *logbook*, week after week of lived experience: the state the van was in, the call made, the money booked, the state it landed in, and the call made there.

SARSA needs nothing else. Maintain a table $q(s, a)$ of *estimates*, current best guesses of $Q^\star(s, a)$, one cell per state-action pair, twelve for the fleet, with every cell starting at 0, as in the visualization. Then, after every lived week, improve the single cell just used.

For a moment, forget the future, and ask the simplest version of the question: what does *one* week of a given pair pay on average, say $(\textsc{worn}, \textsc{run})$? The odds are unknown, but the logbook shows what those weeks actually paid: $+72$, $+72$, $+72$, -208, $+72$, … (an excerpt, mostly profits with the occasional breakdown). Averaging them is the manager's reflex, and there is a bookkeeping trick that needs no filing cabinet: keep one number per cell and, each time a new week arrives, nudge it toward what the week paid:

$$q(s_i, a_i) \;\leftarrow\; q(s_i, a_i) \;+\; \alpha\big[\, r_i - q(s_i, a_i) \,\big]. \tag{6.1}$$

In words: the arrow is an instruction, not an equation; compute the right side and replace the left side by it, moving the estimate a fraction α of the way toward what the week actually paid. The number α is the *learning rate*, 0.15 in the visualization. (α is the Greek letter alpha.) If α shrinks like $1/n$ as the observations accumulate, n counting the weeks of that pair seen so far, these nudges reproduce *exactly* the average of those weeks (two lines of school algebra, deferred to Appendix B); held at a small constant such as 0.15, they produce an average that

[5] Scenes "$Q^\star$: the action scorecard" and "The Bellman equation," interactive in the visualization.

[6] Scene "Why DP doesn't scale" of the interactive visualization prices this out.

gently favors recent weeks. Never overwrite, only nudge: one lucky or unlucky week must not erase a year of evidence.

This formula is already on the manager's desk. It is *exponential smoothing*, the standard forecast-revision update used in demand planning and spreadsheet forecasting: new forecast = old forecast + α (actual − old forecast); revise the plan by a fraction of the miss, and never bet everything on the latest data point. SARSA forecasts the value of a decision the way a planner forecasts sales.

But $Q^\star(s_i, a_i)$, the number the table is after, is not the average of one week's cash alone: by the recurrence (4.3), a week is its cash *plus* γ times the value accumulated from the pair where the van lands, the new state and the call made there. So the week's true "actual" is $r_i + \gamma\, Q^\star(s_{i+1}, a_{i+1})$. The future has not happened yet, and $Q^\star$ is exactly what we do not know, so the table's current best guess $q(s_{i+1}, a_{i+1})$ stands in for it: the same move as plugging a forecasted cash flow into an NPV calculation. One logbook line (s_i, a_i, r_i, s_{i+1}, a_{i+1}) therefore reports the evidence $r_i + \gamma\, q(s_{i+1}, a_{i+1})$, and feeding that richer actual into the smoothing rule (6.1) gives SARSA's update:

$$q(s_i, a_i) \;\leftarrow\; q(s_i, a_i) \;+\; \alpha\big[\, r_i + \gamma\, q(s_{i+1}, a_{i+1}) - q(s_i, a_i) \,\big]. \tag{6.2}$$

In words: the same forecast revision as before, only the "actual" grew up; it is now this week's money plus the discounted value of where the week left you.

Name the bracket: it is the *surprise* (the planner's "miss"), the gap between what this week's evidence says the pair is worth and what the table currently believes. A positive surprise means the week looked better than the table believed; a negative one, worse. To see one nudge land, feed in week one of Example 3.4 with the table still all zeros: the logbook line is (HEALTHY, RUN, +95, WORN, SERVICE); the evidence is $95 + 0.9 \cdot q(\text{WORN}, \text{SERVICE}) = 95 + 0.9 \cdot 0 = 95$; the surprise is $95 - 0 = 95$; and the cell $q(\text{HEALTHY}, \text{RUN})$ moves from 0 to $0 + 0.15 \cdot 95 = 14.25$. One cell moves; the other eleven stay put. Now the punchline. The Bellman equation (5.1) says that $Q^\star(s_i, a_i)$ is the *average* of that very evidence over many weeks, and an average over many executions is exactly how the expectation was defined in Definition 4.3. SARSA computes that average the only way available without the odds: incrementally, with the smoothing recipe of (6.1). Set (6.2) beside (5.1): the update rule is the Bellman equation with the expectation replaced by an arriving average and $Q^\star$ replaced by the current guess q. SARSA solves the Bellman equation by averaging evidence instead of computing expectations.

Remark 6.1. The name SARSA spells the five logbook entries each update consumes: State, Action, Reward, State, Action.

Which calls should be made while the table is still being learned? Mostly the one the table currently favors: at state s, play the action with the largest $q(s, a)$. But in a small random share of the weeks (a fraction $\varepsilon = 0.15$ in the visualization, held constant, like α; ε is the Greek letter epsilon) the call is instead drawn at random, because a cell that is never visited is never improved: you cannot learn the value of servicing if you never service. The a_{i+1} in the update is the action *actually* chosen at s_{i+1} by this very playbook, so SARSA learns the value of what it really does, occasional experiments included.

Run this, week after week, on one endless stream of lived weeks: drive, get billed, nudge one cell. The table drifts toward $Q^\star$, and reading off the best action in each row recovers the playbook. In the visualization the same three bands as in the scorecard of Section 5 (RUN when HEALTHY, SERVICE when WORN, REPLACE when SHAKY or FAILING) emerge within a few hundred weeks of raw experience, the odds never shown.[7] One honest caveat: with α and ε held constant, the twelve numbers themselves stay rough; the experiments cost money, so the

[7] Scene "SARSA: learn from the logbook" of the interactive visualization: watch one week nudge one cell.

cells settle below the $Q^\star$ values and keep jittering. But which call tops each row, and that is all the playbook reads, comes out right.

Closing

These notes modeled sequential decision problems as Markov decision processes, let policies generate random trajectories, scored each trajectory by its cumulative reward, averaged that score to obtain something maximizable, and characterized the best policy through the Q-function and the Bellman equation. When the odds are unknown, SARSA learns the same scorecard from the logbook alone. The fastest way to make all of this concrete is to play: open https://sml-fs26.github.io/classic_rl/repair-or-replace/, run the fleet for a quarter, drag the γ slider, and watch the $Q^\star$ scorecard fill in, scene by scene, in the same order as these notes.

A The Robbins–Monro algorithm

Strip the fleet away and a bare problem remains: find the number $x^\star$ that *equals the average* of noisy evidence, when the average cannot be computed directly and the evidence arrives one sample at a time. The Robbins–Monro algorithm keeps a running guess x and, when the n-th sample arrives, nudges it:

$$x \;\leftarrow\; x + \alpha_n\,(\text{sample} - x).$$

In words: move the guess a fraction α_n of the way toward each sample as it arrives. The step sizes $\alpha_1, \alpha_2, \alpha_3, \ldots$ must satisfy two demands. The nudges must add up without bound, meaning that $\alpha_1 + \alpha_2 + \alpha_3 + \cdots$ grows past any number, so that the iteration can travel from any bad start all the way to $x^\star$. And their squares must add up to something finite, meaning that $\alpha_1^2 + \alpha_2^2 + \alpha_3^2 + \cdots$ stays finite, so that the noise in the samples eventually cancels out instead of jiggling the guess forever. The choice $\alpha_n = 1/n$ satisfies both demands: the running total $1 + 1/2 + 1/3 + \cdots$ climbs past any bound, just ever more slowly (a classical, if surprising, fact), while the squared total $1 + 1/4 + 1/9 + \cdots$ settles below 2.

The SARSA update (6.2) is exactly a Robbins–Monro iteration applied cell by cell to the Bellman equation (5.1): the guess is the cell $q(s_i, a_i)$, and the arriving sample is the evidence $r_i + \gamma\, q(s_{i+1}, a_{i+1})$. With one honest extra: that sample is itself measured with the current table q, so the algorithm pulls itself up by its own bootstraps, and making this rigorous took decades. And a second honest extra: Section 6 holds α constant at 0.15, deliberately failing the second demand, so the noise never fully cancels; that is exactly the jitter conceded there, and Appendix B explains what the constant buys in exchange. The recipe is due to Herbert Robbins and Sutton Monro, who published it in 1951.

B The running average

Appendix A gave the general principle; here is the alternative, elementary explanation, the two lines of school algebra promised in Section 6. Let A_n be the average of the first n observations $x_1, \ldots, x_n$, that is, $A_n = (x_1 + \cdots + x_n)/n$. Then

$$n\,A_n \;=\; x_1 + \cdots + x_n \;=\; (n-1)\,A_{n-1} + x_n.$$

In words: n times the average is the total of all n observations, and that total is the total of the first $n-1$ of them, which is $(n-1)\,A_{n-1}$ (average times count), plus the newest one. Dividing by n and rearranging,

$$A_n \;=\; A_{n-1} + \frac{1}{n}\,(x_n - A_{n-1}).$$

In words: each new observation moves the average by $1/n$ of the surprise, the gap between the newcomer and the average so far. A two-number check: the average of 10 and 16 is 13; a third observation of 22 moves it to $13 + \frac{1}{3}(22 - 13) = 16$, which is indeed the average of 10, 16, and 22.

Now replace the shrinking $1/n$ by a small constant α. The result is an average that gently favors recent observations: each old observation's influence shrinks by a factor $1 - \alpha$ with every new week. That is the right choice here, because the early evidence was measured against a poor table q and should fade. With that one substitution, the running-average recipe *is* the SARSA update (6.2): the rule is nothing more exotic than a running average of the weekly evidence.